

OpenClaw'ın içi

Bir mesaj geldiğinde ne oluyor: sistem prompt nasıl derleniyor, skill'ler nasıl yükleniyor ve neden gövdeleri prompt'a girmiyor, bağlam motoru hangi dört noktada devreye giriyor, bellek nasıl katmanlanıyor, hatırlama neden iki şeride ayrılmış, ve zamanlayıcı neyi tetikliyor. **Hepsi çalışan kuruluma sorularak ölçüldü.**

sürüm • OpenClaw 2026.7.1-2 • kurulu ajan: main • model: claude-opus-4-8 (1.048.576 token pencere)

yöntem • ① canlı gateway'e RPC ② kaynak kodu (7.831 dosya / 1,67M satır) ③ OpenClaw'ın kendisine soruldu

kaynak • docs/concepts/*.md • src/agents/system-prompt.ts • src/gateway/methods/core-descriptors.ts

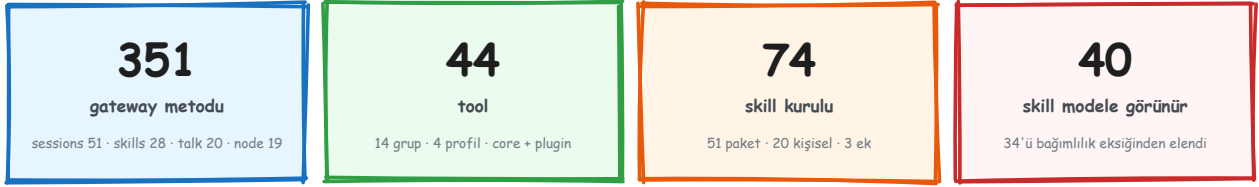
0 · Nasıl ölçüldü

Bu belgedeki hiçbir sayı dokümandan alınmadı. Üçü de ayrı yöntem, ve birbirini doğruluyor.

ETİKET	ANLAMI
ÖLÇÜLDÜ	Çalışan gateway'e RPC atıldı, dönen sayı yazıldı
KAYNAK	Depodaki TypeScript'ten okundu, dosya ve satır verili
AJANDAN	OpenClaw'ın kendisine soruldu; sonra ① veya ② ile doğrulandı

Üçüncü yöntem bu projede yeni. Kendi ajanımıza eklediğimiz `/openclaw` kaçış kapısından OpenClaw'a kendi mimarisi soruldu. Verdiği cevap kaynak koddan doğrulandı — ve bir noktada belgeden daha kesin çıktı: prompt'un **önbellek sınırı**. O ayrımı hiçbir doküman sayfası bu netlikte yazmıyor.

Ölçülen yüzey



ÖLÇÜLDÜ Dört sayı, dört ayrı RPC'den: `core-descriptors.ts` · `tools.catalog` · `skills.status`.

İçindekiler

- 1 Sistem prompt ve önbellek sınırı
- 2 Tool sistemi: 14 grup, 4 profil
- 3 Skill kapısı: 74 → 40
- 4 Skill'in asıl fikri: indeks, gövde değil
- 5 Bağlam motoru: dört yaşam noktası
- 6 Sıkıştırma ve bölme noktası
- 7 Bellek: beş katman
- 8 Hatırlama: iki şerit
- 9 Zamanlayıcı ve olay tetikleyici
- 10 Bizim gateway'le yan yana

1 · Sistem prompt ve önbellek sınırı

AJANDAN **KAYNAK** Prompt her koşuda **yeniden derleniyor** — ama ikiye bölünmüş hâlde, ve bölme yeri maliyetin tamamını belirliyor.

SINIRIN ÜSTÜ — kararlı, prefix-cache'lenir

- Sabit bölümler — Tooling · Execution Bias · Safety · Workspace
- TOOL ŞEMALARI — policy ile süzölmüş her tool'un tam JSON schema'sı
- <available_skills> — indeks: name · description · location · sha256
- MEMORY.md + USER.md — bootstrap sınırıyla (20k/dosya, 60k toplam)
- AGENTS.md · SOUL.md · TOOLS.md · IDENTITY.md · HEARTBEAT.md

CACHE SINIRI

SINIRIN ALTI — her turda değişir

- Messaging · Runtime satırı · Date/Time · output direktifleri
- context engine'in assemble()'dan dönen systemPromptAddition'ı en öne eklenir

Üstteki blok turlar boyunca aynı kaldığı için sağlayıcının **prefix-cache**'ine giriyor; alttaki her turda yeniden yazılıyor. Değişken olan her şeyi aşağı itmek, sabit olanı ucuzlatıyor.

Derleme üç parçadan oluşuyor **AJANDAN** : `buildAgentSystemPrompt` saf bir renderer, `resolveAgentSystemPromptConfig` konfigürasyon düğmelerini çözüyor, runtime adaptörleri canlı gerçekleri topluyor.

Neden önemli: tool şemaları ve skill indeksi **her turda** ödeniyor — ama sabit oldukları için önbellekten geliyor. Bir tool eklemek, o oturumun bütün kalan turlarının önbelleğini bozmuyor; sınırın *altına* değişken bir şey koymak ise hiçbir şeyi bozmuyor. Tasarım bu asimetriyi kullanıyor.

2 · Tool sistemi

ÖLÇÜLDÜ tools.catalog → **44 tool, 14 grup, 4 profil**. Gruplar core ve plugin diye ikiye ayrılıyor; profil hangi grupların açık olacağını belirliyor.

CORE GRUPLARI — 37 tool

fs	4	read write edit apply_patch	runtime	3	exec process code_execution
web	3	web_search web_fetch x_search	memory	2	memory_search memory_get
sessions	7	list history send spawn yield subagents	ui	2	browser canvas
messaging	1	message	automation	3	heartbeat_respond cron gateway
nodes	1	nodes	agents	6	goal plan skill_workshop
media	5	image music video tts			

PLUGIN GRUPLARI — 7 tool

plugin:file-transfer	4	dir_fetch dir_list file_fetch file_w	plugin:tavily	2	tavily_search tavily_extract
plugin:ollama	1	node_inference			

PROFİLLER — minimal · coding · messaging · full (hangi grupların açık olduğunu profil belirliyor)

Tool'lar tek tek değil **grup** hâlinde açılıp kapanıyor, ve profil bir grup kümesi. "Ajanım ne yapabilsin" sorusu 44 kutucuk yerine tek bir profil seçimi.

Bizim için doğrudan sonucu olan bulgu: web grubunda web_search, web_fetch, x_search; plugin:tavily'de tavily_search ve tavily_extract var. Yani **internet araması OpenClaw'da zaten kurulu** — dışarıdan bir arama aracı eklemekten önce bakılacak ilk yer burası.

memory grubundaki memory_search ve memory_get retrieval'ın tool yüzü; §8'de nasıl sıraladığına bakıyoruz.

3 • Skill kapısı

ÖLÇÜLDÜ Diskte **74** skill var, modele **40**'ı görünüyor. Aradaki 34, prompt'a hiç girmiyor.



Kapı yükleme anında işliyor. Bir skill neye ihtiyaç duyduğunu **beyan ediyor**; beyan karşılanmıyorsa liste dışı kalıyor.

Yükleme sırası — altı kaynak, en yüksek öncelik kazanır

ÖNCELİK	KAYNAK	YOL
1 — en yüksek	Workspace skills	<workspace>/skills
2	Proje ajan skill'leri	<workspace>/agents/skills
3	Kişisel ajan skill'leri	~/agents/skills
4	Yönetilen / yerel	<state-dir>/skills
5	Paketle gelen	kurulumla birlikte
6 — en düşük	Ek dizinler + plugin skill'leri	skills.load.extraDirs

Aynı isim birden çok yerde varsa yüksek olan kazanıyor. **SKILL.md** bir kökün altında **6 seviyeye kadar** herhangi bir yerde bulunabiliyor.

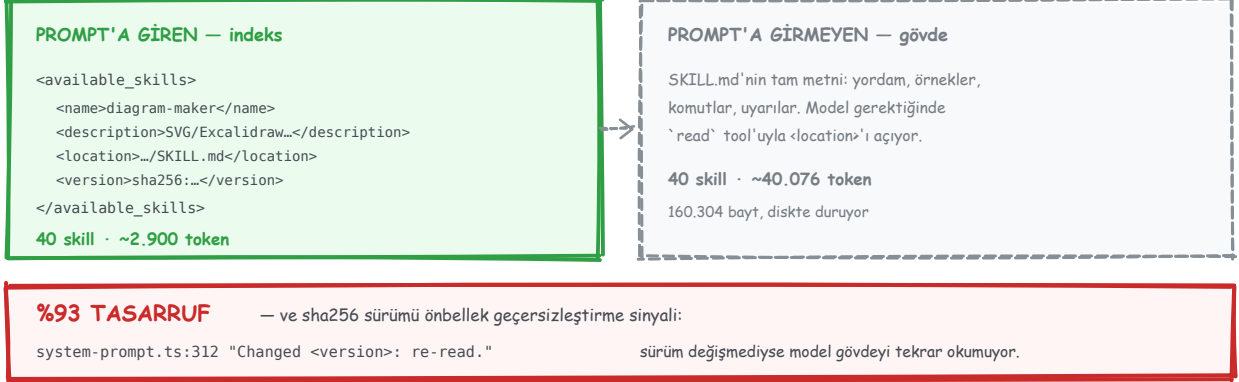
Kapının kendisi

```
metadata: { "openclaw": {
  "requires": { "bins": ["uv"], "env": ["GEMINI_API_KEY"], "config": ["browser.enabled"] },
  "os": ["darwin"],          // platform filtresi
  "always": true             // bütün kapıları atlar
}}
```

Bu kurulumda elenenlerin dağılımı **ÖLÇÜLDÜ**: **6** skill platform uyumsuz (darwin-only), kalan elemelerin tamamı **eksik ikili** — **op**, **gemini**, **whisper**, **mcporter**, **obsidian** ... Yani sistemde kurulu olmayan bir aracın skill'i, modelin dikkatini hiç tüketmiyor.

4 · Skill'in asıl fikri

Bu belgedeki en değerli mekanizma, ve bir cümlede özetlenebilir: **prompt'a skill'in gövdesi değil, indeksi giriyor.**



ÖLÇÜLDÜ 40 skill'in SKILL.md dosyaları diskte **160.304 bayt**. Prompt'a giren indeks **11.584 bayt**.

Modele verilen talimat kaynakta duruyor **KAYNAK** src/agents/system-prompt.ts:310-312 :

```
Scan <available_skills>. Clear match: read exact <location> with `read`; obey.
"Changed <version>: re-read. Several: most specific. None: read none."
```

Dört satırlık bir kayıt — ad, açıklama, **yol** ve **sha256 sürümü**. Model bir eşleşme görürse **read** tool'uyla o yolu açıp gövdeyi kendisi getiriyor.

Sürüm alanı bir önbellek sinyali. "Changed version: re-read" — sha256 değişmediyse model gövdeyi tekrar okumuyor. Yani skill'in içeriği değişmedikçe, onu bir kez okumuş bir oturum bir daha ödemiyo.

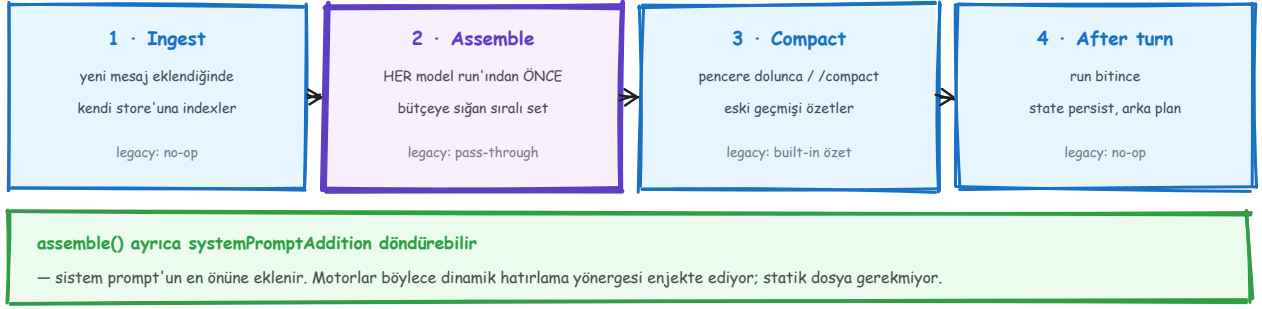
Tool ile skill arasındaki fark

	TOOL	SKILL
Ne	Runtime'a kayıtlı fonksiyon	Markdown yordam (SKILL.md)
Prompt'taki yeri	Tam JSON şema, sürekli mevcut	Yalnız indeks satırı
Nasıl çalışır	Model çağırır, runtime yürütür	Model okur, talimatı izler
Ne öğretir	Ne yapılabileceğini	Ne zaman ve nasıl yapılacağını
Maliyeti	Şema boyu × her tur	~4 satır × her tur, gövde talep üzerine

Kısaca: **tool bir yetenek, skill o yeteneğin kullanma kılavuzu.** Bir skill genellikle mevcut tool'ları hangi sırayla çağıracağını anlatıyor; yeni bir yetenek eklemiyor.

5 • Bağlam motoru

KAYNAK Takılabilir bir arayüz: her model koşusunda **dört** yaşam noktasında devreye giriyor.



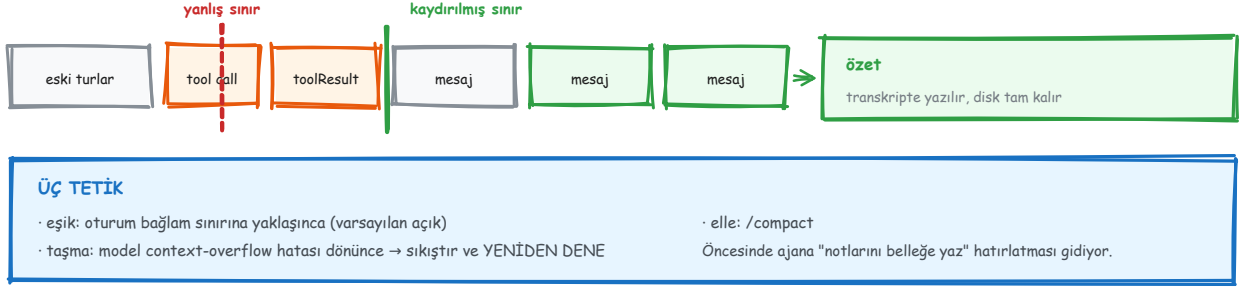
Bu kurulumda **legacy** motor açık: ingest ve after-turn hiçbir şey yapmıyor, assemble mevcut sanitize→validate→limit hattına geçiyor, compact yerleşik özetlemeye devrediyor. Asıl "bağlam mühendisliği" **assemble**'da.

Opsiyonel kancalar: `bootstrap`, `maintain` (transkripti güvenli yeniden yazma), `prepareSubagentSpawn` / `onSubagentEnded`. `turnMaintenanceMode: "background"` ile bakım işi cevabı bloklamadan arkada koşuyor.

Bizim `context_engine.py` 'mizle aynı fikir — bütçeye göre mesaj seçen bir katman. Farkı, OpenClaw'ının **eklenti yuvası** olması: `plugins.slots.contextEngine` ile motorun tamamı değiştirilebiliyor. Bizde tek bir sınıf var ve değiştirmek kod değişikliği demek.

6 · Sıkıştırma

Eski turlar özetleniyor, yeniler olduğu gibi kalıyor. İlginç olan **bölme noktasının nereye konmadığı**.



Bölme noktası bir tool bloğunun içine düşerse OpenClaw sınırı **kaydırıyor**, çünkü bir tool çağrısı sonucundan ayrılırsa geride anlamsız bir yarım kayıt kalır.

Bu bizde de var ve bağımsız olarak aynı sonuca varmışız: `context_engine._safe_boundary()` bir tool çağrısını sonucundan asla ayırmıyor. İki ayrı proje aynı tuzağa çarpıp aynı çözümü bulmuş — bu, kuralın tesadüf olmadığının işareti.

Sıkıştırma öncesinde ajana **"önemli notlarını belleğe yaz"** hatırlatması gidiyor; bağlam kaybını böyle önüyor. Tam geçmiş diskte kalıyor — sıkıştırma yalnız modelin bir sonraki turda *gördüğünü* değiştiriyor.

Yeni konfigürasyonlarda varsayılan mod **"safeguard"**: daha sıkı korumalar ve **özet kalitesi denetimi**. Eski davranış için `mode: "default"` açıkça yazılmalı.

7 • Bellek: beş katman

KAYNAK docs/concepts/memory-architecture.md — beş katman, ve hangisinin prompt'a girdiği katmanın kimliğiyle belirlenmiş.

KATMAN	YÜZEY	KİM YAZAR	PROMPT'A GİRER Mİ
Instructions	AGENTS.md, workspace	yalnız insan	HER ZAMAN
Curated core	MEMORY.md, USER.md	dreaming; kullanıcı	HER ZAMAN, bütçeli
Episodic	memory/YYYY-MM-DD.md	ajan; transkript	ASLA — aranabilir
Prospective	standing intents, cron	intent tool	tetik ateşlerse
Review	DREAMS.md	dreaming	ASLA — insan için

ASIL SINIR — curated ile episodic arasında

Otomatik enjeksiyon YALNIZ curated katmandan. Günlük notlar ve transkriptler, eşleşme ne kadar güçlü olursa olsun asla enjekte edilmiyor — bu bir ayar tercihi değil, güvenlik özelliği.

Curated katman küçük, hep bağlamda ve yalnız kapılı birleştirmeye yazılıyor. Episodic katman büyük, eklemeye açık ve **yalnız arama** ile erişilebilir.

Mimarının beş kuralı

- Gizli durum yok.** Model yalnız çalışma alanındaki dosyalara yazılanı hatırlıyor; her bellek yüzeyi bir metin düzenleyiciyle okunup değiştirilebiliyor.
- Zor olan yazmak.** Not dosyaları üzerinde retrieval, çok daha ağır tasarımlarla yarışabiliyor; bellek sistemlerini bozan şey **yazma anındaki güvenilmez seçim**. O yüzden seçim, cevap yolundan alınıp arka plan geçişine taşınmış.
- Güvenlik sınırı yazma yolunda.** Zehirlenmiş bilgiyi içerik taramasıyla yakalamak güvenilmez; OpenClaw bunun yerine **kaynak (provenance)** zorunlu kılıyor ve terfiyi yapısal olarak kapılıyor.
- Deterministik kapı, içinde model yargısı.** Skorlama, eşik, uygunluk ve yaşam döngüsü deterministik kod; model yalnız dil yargısı gerçekten gerektiğinde ve kodun çizdiği sınırın içinde.
- Arıza cevabı bloklamaz.** Cevap yolundaki her bellek adımının zaman aşımı, yedeği ya da ikisi var. Çöken bir bellek alt sistemi hatırlama kalitesini düşürüyor; turu **yemiyor**.

8 • Hatırlama: iki şerit

Ayrım **maliyete göre** yapılmış: varsayılan şerit hiç model çağırıyor.

ŞERİT 1 — hep açık, SIFIR model çağırısı · Bootstrap: MEMORY.md + USER.md oturum başında, her turda tazelenir · Sıralı arama: hibrit alaka × üstel tazelik (30 gün yarı ömür) × önem (1-10) · Tetik enjeksiyonu: yazarların ilıstirdiđi tetik ifadeleri ön-elemeden geçer; skor ≥ 0.72 → turda en fazla 3 giriş Önem yazma anında verildiđi için sorgu anında model çağırısı gerekmiyor.	ŞERİT 2 — tırmanma Gerçek bir alt-ajan turu: konuşma geçmişinde arama ve okuma yapılabilir, konuşmalar arası transkript hatırlama dahil. Pahalı, o yüzden yalnız gereken turlarda çalışıyor. Bu ayrım maliyete göre yapılmış: varsayılan şerit gecikme eklemiyor.
--	--

Önem (1-10) **yazma anında** veriliyor — o sırada zaten döngüde bir model var. Sorgu anında model çağırısına gerek kalmamasının sebebi bu.

Sıralama formülü

$$\text{skor} = \text{hibrit_alaka} \times \text{exp_tazelik}(30 \text{ gün yarı ömür}) \times \text{önem}(1-10)$$

Hibrit = gömme (embedding) + anahtar kelime. Sağlayıcı seçilebilir: OpenAI (varsayılan), Gemini, Voyage, Mistral, Bedrock, **local**, Ollama, LM Studio.

Tetik enjeksiyonu – ve neden yalnız curated'dan

- Keep the gateway on loopback. <!-- trigger: gateway setup, network safety --> <!-- importance: 9 -->

Gelen her mesaj bu tetik ifadelerine karşı hızlı bir sözlüksel + vektör ön-elemesinden geçiyor; skoru **0.72** ve üstü olanlar gizli bir bağlam blođu olarak, **turda en fazla üç tane** enjekte ediliyor.

Ve şu kısıt bir ayar deđil: otomatik enjeksiyon yalnız curated katmandan yapılıyor. Günlük notlar ve transkriptler eşleşme ne kadar güçlü olursa olsun **asla** enjekte edilmiyor. Gerekçe doğrudan yazılmış: *"bu bir güvenlik özelliđi, bir ayar tercihi deđil — sıradan turlarda denetlenmemiş içeriđi prompt'un dışında tutuyor."*

9 • Zamanlayıcı

ÖLÇÜLDÜ `cron.status` → etkin, SQLite'ta saklanıyor (`~/openclaw/state/openclaw.sqlite`), şu an **0** iş. Beş zamanlama tipi var.

<code>at</code>	tek seferlik zaman damgası	ISO 8601 ya da `20m`
<code>every</code>	sabit aralık	10m · 1h · 1d
<code>cron</code>	5 ya da 6 alanlı ifade	--tz ile IANA saat dilimi
<code>on-exit</code>	izlenen komut çıkınca	tur yıkımından sağ çıkar
<code>stream</code>	uzun ömürlü komuttan	toplu satırlardan tetiklenir

OLAY TETİKLEYİCİ — koşul gözcüsü

`every/cron/stream`'e başsız bir koşul betiği eklenir. Zamanlayıcı asıl yükü YALNIZ betik `fire:true` dönerse koşturuyor. Yani zamanlama ile karar ayrı: "her 30 sn bak, ama sadece değiştiyse uyandır."

Tepe saat ifadeleri yük yığılmasını azaltmak için 5 dakikaya kadar otomatik kaydırılıyor (--exact ile kapatılır).

CLI'da `openclaw automations`; `openclaw cron` aynı komutların takma adı olarak duruyor.

En öğretici parça olay tetikleyici. Zamanlama ile *karar* ayrılmış: `every 30s` deyip koşul betiği `fire:true` dönmedikçe asıl yükü çalıştırmıyor. Yani "her 30 saniyede bir bak, ama sadece **değiştiyse** ajanı uyandır." Yoklama maliyeti bir betik; ajan turu maliyeti yalnız gerçekten bir şey olduğunda.

Yük dağıtımı için küçük bir ayrıntı: tepe saat ifadeleri (`0 * * * *` gibi) otomatik olarak **5 dakikaya kadar kaydırılıyor**. Kesin zamanlama gerekiyorsa `--exact`, kendi penceren için `--stagger 30s`.

10 · Bizim gateway'le yan yana

Mimariyi OpenClaw'dan aldık; hangi parçayı ne kadar aldığımız artık ölçülebilir.

MEKANİZMA	OPENCLAW	BİZİM GATEWAY
Oturum modeli	agent:main:whatsapp:group:...	✓ aynı şema — agent:main:web:dm:local
Hook noktaları	13	✓ 13 — aynı adlar, aynı terminal anahtarları
Bağlam motoru	Eklenti yuvası + 4 yaşam noktası	△ tek sınıf, yuva yok
Sıkıştırma sınırı	Tool çiftini ayırmıyor	✓ aynı kural — _safe_boundary()
Bellek	5 katman · dreaming · terfi kapıları	△ 2 katman — MEMORY.md + günlük notlar, dreaming yok
Retrieval	Hibrit + tazelik + önem, tetik enjeksiyonu	✗ yok — TF-IDF yalnız belge aramada
Skill	74 kurulu · indeks + talep üzerine gövde	✗ yok
Tool grupları / profil	14 grup · 4 profil	△ düz liste (16 tool), profil yok
Zamanlayıcı	5 tip + koşul gözcüsü, SQLite	△ cron.py yazıldı, tick() hiçbir yere bağlı değil
Onay kapısı	Ayrı scope'lar, permissions.respond	✓ tek-seferlik digest izinli kapı

Buradan çıkan üç iş

- Skill indeksi deseni.** Bizim ajanımız 16 tool şemasını her turda ödüyor ve "merhaba" bile **4.601 token**. OpenClaw'ın **%93**'lük tasarrufu aynı fikirle geliyor: gövdeyi değil indeksi taşı. Doğrudan uygulanabilir.
- Tool profilleri.** 44 tool'u profil altında toplamak, bizim **OPENCLAW_TOOLS** filtresinin genel hâli.
- Önem × tazelik sıralaması.** Yazma anında önem vermek, sorgu anında model çağrısını gereksiz kılıyor — bizim bellek katmanımızda karşılığı yok.

Ölçüm tarihi 2026-08-17 · OpenClaw 2026.7.1-2 · Sayılar bu kurulumla aittir; başka bir makinede skill ve tool sayıları farklı çıkar. Yöntem tekrarlanabilir: `openclaw gateway call skills.status --json` · `openclaw gateway call tools.catalog --json`